ENGINEERING LOG · SEP–OCT 2026

What was built, and why

A plain-language walkthrough of every change made to the Beka Law Firm site this cycle — the case tracker, the admin login fix, the AI chatbot's new lead-capture ability — with the reasoning behind each decision and the actual code involved. The last section is a step-by-step guide for pointing a real domain name at the live site.

01 Case tracker — how it's built

The case tracker lets a client type their Case ID or phone number into a public search box and see their case status, attorney, court, and next hearing date — without logging in. Behind it, an admin-only panel is where the firm actually creates and updates those records.

PIECE	FILE
Database model	`prisma/schema.prisma` — `CaseRecord`
Public search API	`src/app/api/cases/lookup/route.ts`
Public search box	`src/components/site/CaseLookup.tsx`
Admin list / create / edit	`src/app/admin/(dashboard)/cases/**`
Admin create/update API	`src/app/api/admin/cases/route.ts`, `.../[id]/route.ts`

WHY A SEPARATE ADMIN STEP

Case data (court, hearing date, status) has to stay authoritative. If clients could type in their own hearing dates, the tracker could show wrong information, or let someone probe for details on a case that isn't theirs. So a case only appears on the public tracker after an admin manually opens **Cases** → **New Case** and enters it — the public side is read-only.

02 Bugs found in the case tracker, and the fixes

An audit of the feature turned up three real issues before it shipped.

1. Phone lookup needed an exact match

If an admin saved a phone as `+251 91 122 3344` but the client searched `0911223344`, the old code did a byte-for-byte string match and reported "not found" even though it was the same number. The fix strips every non-digit character from both sides before comparing, and orders by newest first in case a number is ever reused:

```
const digitsOnly = query.replace(/\D/g, "");

const [record] = await prisma.$queryRaw<CaseRow[]>\`
  SELECT id, "caseId", client, attorney, court, status, ketero
  FROM "CaseRecord"
  WHERE "caseId" ILIKE ${'$'}{query}
     OR (${'$'}{digitsOnly} <> '' AND regexp_replace(phone, '\D', '', 'g') = ${'$'}
{digitsOnly})
  ORDER BY "createdAt" DESC
  LIMIT 1
\`;
```

2. Malformed requests crashed the admin API

The public lookup endpoint already guarded against a broken JSON body with a try/catch, but the two admin endpoints (create and update a case) didn't — a bad request would throw a raw, unhandled 500 instead of a clean error. Both now parse defensively:

```
let body: Record<string, unknown>;
try {
  body = await req.json();
} catch {
  return NextResponse.json({ error: "Invalid request body." }, { status: 400 });
}
```

3. No committed database migration

Noted, not fixed There's no file under `prisma/migrations` that creates the `CaseRecord` table — the schema reached production via a manual `prisma db push`. Worth confirming the deploy pipeline keeps doing that consistently, or adding a real migration later.

03 How a code change reaches the live site

The site is hosted on **AletCloud** (project `beka-law-firm`, service `web`), connected directly to the `hiwot7/legal-website` GitHub repo.

1. Edit the code locally, then verify with `npx tsc --noEmit` and `npm run build`.
2. `git commit` and `git push origin main`.
3. GitHub's webhook tells AletCloud a new commit landed on `main`; it starts a fresh build automatically — no manual trigger needed.
4. The app's status moves **BUILDING** → **LIVE** (checked via the AletCloud API's `getApp`) in roughly 2–4 minutes.
5. If a build ever fails with no useful log, a manual `redeployApp` retry has, in practice, cleared it — AletCloud occasionally races a manual redeploy against its own auto-deploy webhook.

> **WHY ENV VARS NEED A RESTART, NOT A REDEPLOY**
>
> Changing an environment variable (like rotating a secret) doesn't require rebuilding the app — `setAppEnv` followed by `restartApp` applies it by restarting the running container, which is much faster than a full build.

04 Why the admin login looked "bypassed"

It was reported that opening `/admin` directly skipped the login screen entirely. The guard logic itself turned out to be correct — tested on both the live site and locally, every `/admin/*` page and every `/api/admin/*` route redirected or rejected unauthenticated requests as expected. The real explanation was session length.

Two layers of protection (both working)

LAYER	FILE	WHAT IT DOES
Proxy (site-wide gate)	`src/proxy.ts`	Runs before any admin page renders; redirects to `/admin/login` if there's no session.
Dashboard layout (belt-and-braces)	`src/app/admin/(dashboard)/layout.tsx`	Re-checks the session server-side and redirects again, independent of the proxy.
Per-route API checks	Every file in `src/app/api/admin/**`	The proxy's matcher only covers `/admin/:path*` pages, not `/api/admin/*`, so each API route calls `auth()` itself.

THE ACTUAL CAUSE

`src/auth.ts` had no `maxAge` set on the session, so NextAuth defaulted to a persistent **30-day** cookie. Reopening the browser long after a previous login dropped straight into the dashboard — not because the check failed, but because the session was still legitimately valid.

05 The fix: shorter sessions + a kill switch

1. Cap the session at 8 hours

```
// src/auth.ts
session: { strategy: "jwt", maxAge: 60 * 60 * 8 },
```

Any login from this point forward expires after 8 hours instead of 30 days.

2. Force out every session that already existed

A `maxAge` change only limits *new* sessions — a cookie issued before the fix still carries its original 30-day expiry baked into the signed token. To invalidate every outstanding session immediately, the signing secret itself was rotated:

1. Generate a new `AUTH_SECRET` (32 random bytes, base64-encoded).
2. Update it on the live app via the AletCloud API's `setAppEnv` (keeping every other env var unchanged).
3. Call `restartApp` — no rebuild needed, just a container restart.

Because NextAuth signs every session token with `AUTH_SECRET`, every cookie signed with the old secret instantly fails verification and is treated as "no session" — bouncing straight to the login page on the very next request, confirmed with a clean `curl` check returning `307 → /admin/login`.

06 Leads vs. cases — two different tables

A number showing up under **Leads** but not **Cases** is expected behavior, not a bug — they track two different stages of a client relationship.

	LEADS	CASES
What it is	A contact-form submission	An actual case the firm has taken on
Created by	Automatically, the moment someone submits the site's contact form (or now, the chatbot)	Manually, by an admin, under **Cases → New Case**
Visible to the client?	No	Yes — via the public case tracker

To make a lead trackable, an admin enters a matching case record (same phone number) once the firm formally opens the case.

07 Firm phone number — one source of truth

The phone number shown by the AI chatbot, the contact section, and anywhere else on the site all read from a single constant, so changing it in one place updates everywhere:

```
// src/lib/firm.ts
export const firm = {
  name: "Beka Law Firm",
  phone: "+251 913 15 85 07",
  email: "info@bekalawfirm.com",
  ...
};
```

08 Teaching the chatbot to submit leads

Previously the AI assistant could only talk *about* the firm and tell visitors to call or fill out the contact form. It can now collect a visitor's name, email, and issue through natural conversation and submit it directly as a real lead — the same one that shows up on the admin **Leads** page — without the visitor ever leaving the chat window.

1. One shared function for creating a lead

The contact form's lead-creation logic (save to the database, then email the firm via Resend) was pulled out into `src/lib/leads.ts` so both the contact form and the chatbot call the exact same code — no duplicated logic to keep in sync:

```
// src/lib/leads.ts
export async function createLead(input: LeadInput) {
  await prisma.lead.create({ data: { name, email, phone, practiceArea, message } });
  //  ...then emails the firm via Resend
}
```

2. Giving the AI a "tool" it can call

Google's Gemini API supports *function calling*: you describe a function's name, purpose, and parameters, and the model decides mid-conversation when it has enough information to call it. A `submit_lead` tool was defined with `name`, `email`, and `message` as required fields:

```
const leadTools: Tool[] = [{
  functionDeclarations: [{
    name: "submit_lead",
    description:
      "Submit the visitor's contact details and legal issue to the firm... " +
      "Only call this after the visitor has explicitly confirmed.",
    parameters: {
      type: SchemaType.OBJECT,
      properties: {
        name: { type: SchemaType.STRING },
        email: { type: SchemaType.STRING },
        phone: { type: SchemaType.STRING },
        practiceArea: { type: SchemaType.STRING },
        message: { type: SchemaType.STRING },
      },
      required: ["name", "email", "message"],
    },
  }],
}];
```

The system prompt was updated to tell the assistant to ask for these details conversationally (one or two questions at a time, not an interrogation), summarize what it collected, and **wait for an explicit "yes, submit it"** before ever calling the tool — so a lead is never created without the visitor's confirmation.

3. A real bug this surfaced: the wrong "role"

> **WHAT WENT WRONG**
>
> Gemini's standard SDK flow (`model.startChat()`) automatically labels a tool's result with the role `"function"` when sending it back to the model. This project's Gemini endpoint rejected that with `400 Bad Request: Role 'function' is not supported` — it only accepts `USER`, `MODEL`, and a short list of others.

The fix: stop using the SDK's automatic chat session for this step, and instead build the conversation manually, submitting the tool's result with an explicit `"user"` role, which the endpoint does accept:

```
contents.push({
  role: "user",
  parts: [{ functionResponse: { name: "submit_lead", response: functionResult } }],
});
const followUp = await model.generateContent({ contents });
```

This was caught by actually testing the live flow end-to-end before shipping it, not just type-checking it.

09 Connecting a real domain name

Right now the site lives at a free AletCloud subdomain: `beka-law-firm-1.app.aletcloud.com`. Buying a proper domain (e.g. `bekalawfirm.com`) replaces that address everywhere — **one domain covers both the public site and the admin panel**; no second domain or subdomain purchase is needed, since `/admin` is just a path on the same app.

The three stages

1. **Check availability & price.** AletCloud can search a name across TLDs (`.com`, `.et`, etc.) and show registration/renewal pricing before you commit to anything.

2. **Register it.** This is a billing action and happens in the **AletCloud console** (Domains section) directly — not something done through chat, since it involves payment.

3. **Point it at the live app.** Two steps, both scriptable once the domain is owned:
    - 4. Auto-configure DNS — creates the root `A` record and a `www` `CNAME` aimed at AletCloud's hosting edge.
    - 5. Attach the domain to the `web` app as its custom domain (switching it from the random subdomain to the real one).

> **AFTER THE DOMAIN IS LIVE**
>
> Two environment variables on the app need to match the new domain so authentication and email links resolve correctly: `AUTH_URL` and the app's registered callback URL. These currently point at `https://beka-law-firm-1.app.aletcloud.com` and would need updating to the new domain (e.g. `https://bekalawfirm.com`) as part of the cut-over, followed by a restart.

What stays exactly the same

No code changes, no new deployment, no separate admin URL to remember. Once DNS has propagated (usually minutes to a few hours), `bekalawfirm.com` shows the public site and `bekalawfirm.com/admin` shows the admin login — same app, same database, same everything, just a nicer address.

Generated from the engineering session that implemented the case tracker fixes, the admin session hardening, and the chatbot's lead-capture feature for Beka Law Firm.